CSE113: Parallel Programming

Homework 1: Instruction Level Parallelism and C++ Threads
Assigned: September 29, 2026
Due: October 9, 2026

Preliminaries

1. This assignment requires the following programs: `make`, `bash`, `python3`, and `g++`. Please use the Docker image provided for this course to ensure that your code runs in the same environment used by the teaching team for grading. More instructions on how to use the docker image are provided at the end of this document.

2. If you need help, please visit office or mentoring hours. You can also ask questions on Piazza. You are allowed to ask your classmates questions about frameworks and languages (e.g. Docker, Git, and C++). You are not allowed to discuss homework solution details with them.

3. In this assignment, you will be provided with a template code that you will need to complete. You will be graded on the correctness of your code, the performance of your code, and the quality of your report.

4. **Your submission will consist of two items**: A PDF report, and the completed codes. Zip your code and PDF report into a single file and submit it to Canvas.

0.1 Structure of the report:

Create one PDF containing 3 sections, one for each part of the assignment. For each part of the assignment, your report should contain the following sub-sections:

- **Design**: describe the design of your code/solution in enough detail that a knowledgeable programmer could duplicate your work. This includes descriptions of the workflow of each part and a description of each function with its purpose, inputs, outputs, and assumptions it makes about inputs or outputs.

- **Results**: Include your timing results. Present your results in a line graph, where the x-axis is the parameter you are varying (e.g. unroll factor or number of threads) and the y-axis is the speedup relative to the reference implementation. Read each part of the assignment for more details on what to include in your results.

- **Analysis**: Analyze your results. Why do you think you got the results you did? What trend do you observe? Any unexpected results? Does the result agree with what you expected? If not, what could be the reason?

If you want help with your code, you have to share your **Design** part of the report with us. We can't debug your code without knowing what you are trying to do.

There is a template for the report provided in the assignment named **report.tex**. You can use the template to structure your report but you are not required to use it.

1 Loop Unrolling Independent Iterations for ILP (33 points)

Here we will consider unrolling `for` loops that contain independent iterations. However, each iteration will contain a chain of *dependent* instructions. This chain is intended to impede the processor's ability to utilize ILP, either through pipelining or superscalar components. The length of dependent instruction chains is a parameter of the program. Your assignment is to unroll the loops, first executing each iteration sequentially, and then interleaving the instructions from different iterations. Interleaving instructions should allow the processor to exploit more ILP, which should be seen in timing experiments. You will measure the execution time of various dependent chain lengths and unrolling factors.

This assignment is based on a C++ loop structure, parameterized by a dependency chain length N, that looks like this:

```
void loop(float *a, int size) {
  for (int i = 0; i < size; i++) {
     float tmp = a[i];
     tmp += 1.0f;
     tmp += 2.0f;
     tmp += 3.0f;
     // and many more
     tmp += N;
     a[i] = tmp;
  }
}
```

A few things to note about this loop: the dependency chain cannot be re-ordered or statically pre-computed in the compiler because floating point operations must be done in order (i.e. they are non-associative). So the compiler must produce an ISA instruction for each of the addition operations. The code will generate as many addition operations as specified by the chain length N.

In the skeleton provided, you will find a Python code that performs the following:

1. it generates the reference loop
2. it generates stubs for the functions you need to implement
3. puts these functions in a C++ wrapper that will print timing information about the execution of the functions.
4. it compiles the code
5. it runs the code

The Python script takes two command-line arguments: the length of the dependency chain and the unroll factor. Run with `-h` to view the argument specification. You can view the generated C++ file in `homework.cpp` (it will change each time you run the Python script). You can specify various chain lengths to see how the reference loop changes. The unroll factor currently does nothing because that is your job to implement. Initially, your C++ code will compile and run, but it will not be correct, as the two loops that you are supposed to implement contain empty bodies. Once you implement the functions, the C++ code will report the speedups that unrolled loops provide.

1.1 Technical notes

- The coding aspect of this assignment is constrained entirely to `part1/part1.py`.
- Read through the entire skeleton code to understand the structure. The Python code writes a C++ file that is then compiled with `g++` and executed. The C++ file will time your implementation loop against a reference.
- You can assume that the **size is always a power of 2, and so is the unroll factor**. That is, you do not need to implement "clean up" iterations.
- Remember that floating point constants need a `f` character, otherwise they will be considered double type, which will mess up your timings! For example, the floating point of `2` is `2.0f`
- For your submission, the only file that will be graded is `part1.py`. You can change code in `main.py` while testing, but please ensure your code works with the original version when submitting.

1.2 What to Submit

You will submit a completed `part1/part1.py` file. Run your program with a dependency chain length of 64, and then with unroll factors of lengths 1, 2, 4, 8, 16, 32, and 64. In your report, present your results as a line graph where the unroll factor is the x-axis and the speedup of your two loops (relative to the reference) is the y-axis. Run each experiment at least 5 times and report the average.

Write a few paragraphs explaining the results and how they relate to what we have been learning.

Grading Rubric: The coding part is worth 18 points, and the report is worth 15 points. We will consider both the correctness of your code and performance of your code. Your grade will be based on 4 criteria:

- Correctness: do your functions compute the right result? Your implementation should produce the exact same computations as the reference loop (maybe in different orders but the values of each element of the array should be the same). If not, we cannot grade the rest of the code.
- Conceptual: do your functions actually unroll and interleave instructions?
- Performance: do your performance results match roughly what they should? (On my machine, the interleaved version is about 3x faster than the sequential unrolled version, which is about 1.2x faster than the reference loop when running with a dependency chain length of 16 and an unroll factor of 8. Depending on your machine, this may vary, but you should see a similar trend.)

- Explanation: do you explain your results accurately based on our lectures?

2 Unrolling Reduction Loops for ILP (33 points)

Part 2 is identical to part 1, except we are targeting a different type of `for` loop. Here we are targeting reduction loops, where each iteration depends on the previous one. Recall in lecture that we showed that these loops can still be unrolled to exploit ILP. The loop should be unrolled so that each iteration computes several partial reductions. At the end of the function, there needs to be a loop adding up the totals for each N.

There is only one parameter in this part, the unroll factor. Your timing results can be displayed as a line graph where the x-axis is the unroll factor (or partitions in the code), and the y-axis is the speedup relative to the reference. You only have to go up to an unrolling factor of size 16. Think about different ways to compute the partial reductions and try different approaches. Include graphs and document the different strategies you try.

Again you can assume **the size and unroll factor are always a power of 2**. All other aspects of this part can are the same as part 1.

Performance improvement hint: Try to do less expensive operations inside the loop. For example addition is much faster than multiplication.

Array size note: In order to get meaningful timings, the code uses a very large array. This may overwhelm the machine you are running on. You can tell if this is happening if your code is taking longer than a few seconds to run.

To help with this, please try closing all programs before running your experiments (especially if you are running chrome). If you still don't have enough memory, you can reduce the size of the array, using the `SIZE` constant in the code. If you do this, please try to do your timing experiments with as large of an array as possible on your machine. Also, make sure to reduce your SIZE by a power of two (e.g. change 512 to 64).

Whether or not you reduce your array, your submission must run correctly, e.g. give the correct result, for the original size.

2.1 What to Submit

This is the same as in Part 1. You will submit a completed `part2/part2.py` file. The report should include a line graph with unroll lengths of 1, 2, 4, 8, and 16 in result section.

The coding part is worth 18 points, and the report is worth 15 points.

3 SPMD Parallel Programming Using C++ Threads (34 points)

Here you will get some experience writing parallel code using C++ threads. The file you will complete is `part3.cpp`.

The code creates 3 integer arrays of size 1024, called a, b and c, and are initialized to 0.

Your job is to store the value 1048576 (let's call it K) in each element of each array, but there is a catch. You can only operate on the arrays using the increment operator (i.e. `a[i]++`). Furthermore, each array must be operated in a different specified function (see the code). Notice that the functions use the `volatile` keyword in the argument list. This is so that all of the memory operations actually perform their loads and stores. You should be able to tell from your timing results if your memory operations are being optimized by the compiler.

1. For array a, you must do this computation sequentially in the function `sequential_increment`.
2. For array b, you must do this computation in parallel by modifying the function `round_robin_increment` as an SPMD parallel function using C++ threads. In this function, threads access elements in a round-robin style. That is, a thread with thread id of tid must compute the elements at index i where

   $$i\%NUM_THREADS = tid$$

   a thread must complete all increment operations on a location before moving on to the next location. It must operate on its locations in order, from least to greatest.
3. For array c, you must do this computation in parallel by modifying the function `custom_increment` as an SPMD parallel function using C++ threads. However, you can assign the data to the different threads in any way you'd like. Think about how you can do this in a more efficient way than what is done for array b.

   As a clarification: you must make sure that every increment operation goes to memory. You cannot load the value into a local variable and increment the variable. You must do all of the additions to a single location before moving to the next one.

The code contains timing code. Please print out the timing information and pay attention to the following timings:

- the computation of a related to b
- the computation of a related to c
- the computation of b related to c

3.1 What to Submit

You will submit a completed `part3/part3.cpp` file.

Similar to the previous two problems, part of your submission will be some timing experiments. Please run your code locally with 1, 2, 4, 8, 16 threads (this can be changed with a variable in `main.cpp`). Provide a line graph illustrating your results. The x-axis is the number of threads and the y-axis is the speedup relative to the computation of a (you will have two of these lines).

Please describe your strategy for computing c in the solution section of your report and explain your results in the analysis section.

Also, you may need to increase the size of K to get meaningful performance results. You should try to select a size on your machine where the computation of a takes around 1 second. Please include this number in your report.

The coding part is worth 25 points, and the report is worth 8 points. Your grade will be based on 4 criteria:

- Correctness: do your functions compute the right result? If not, we cannot grade the rest of the code.
- Conceptual: do your functions actually implement the SPMD programming model? is array b computed under the right constraints?
- Performance: do your performance results match roughly what they should? Could you identify ways to improve the performance of array c compared to b?
- Explanation: do you explain your results accurately based on our lectures?

4 Docker Helper

Your homework must run in the course Docker image so that it uses the same environment as used by the teaching team. The image provides a simple Ubuntu environment with `python3`, C++ compilers, `make`, `vim`, and `emacs`. Even if the assignment runs on your Linux, macOS, or WSL installation, test it in the course image before submitting; code that does not run in the image cannot be graded.

4.1 Install Docker

Install and start Docker before continuing. Linux users should follow the official Docker Engine installation guide for Linux: https://docs.docker.com/engine/install/. macOS users should follow https://docs.docker.com/desktop/setup/install/mac-install/, and Windows users should follow https://docs.docker.com/desktop/setup/install/windows-install/.

Verify that Docker is running with:

```
docker --version
docker run --rm hello-world
```

4.2 Run the Course Container

First download the course image:

```
docker pull reeselevine/cse113:latest
```

From the top-level directory of your assignment, start a container and mount that directory at `/assignments`. The mounted directory keeps your assignment files on your computer, so edits made inside or outside the container are visible in both places.

On Linux, macOS, or WSL, run:

```
docker run --name cse113 -v "$(pwd)":/assignments -it reeselevine/cse113:latest bash
```

In Windows PowerShell, run:

```
docker run --name cse113 -v ${pwd}:/assignments -it reeselevine/cse113:latest bash
```

In Windows Command Prompt, run:

```
docker run --name cse113 -v %cd%:/assignments -it reeselevine/cse113:latest bash
```

Inside the container, use `cd /assignments` to access the assignment files. Run the commands specified in each part of the homework, and type `exit` when you are finished.

After the first run, reopen the same container with:

```
docker start -ai cse113
```

To see existing containers, use `docker ps -a`. Do not store important work only inside a container—only files in the mounted `/assignments` directory are kept with your homework on the host machine.